

Propagazione parallela di regole logiche pesate su GPU

Relazione del progetto ArcPal – Architetture Parallele

Alessandro Minisini

25 giugno 2026

Indice

1 Introduzione

Il progetto ArcPal implementa, in CUDA, un *propagatore parallelo* per un frammento di linguaggio logico basato su regole pesate. Il problema è quello tipico della propagazione proposizionale: data una base di regole e un'assegnazione parziale di verità, si vuole estendere l'assegnazione applicando ripetutamente le regole di inferenza, fino a raggiungere un punto fisso in cui nessuna ulteriore deduzione è possibile, oppure fino a rilevare una contraddizione.

Il vincolo principale imposto dalla specifica del progetto (riportata nel documento `tema.pdf` fornito con l'assegnazione) è che il programma debba scalare con istanze di dimensione arbitraria, limitate solo dalla memoria disponibile sulla GPU. Questo ha guidato sia la scelta delle strutture dati (rappresentazioni compatte e contigue, trasferibili in blocco sul device) sia l'esplorazione di diverse strategie di parallelizzazione, che vanno dal semplice ri-lancio iterativo di un kernel fino all'uso di *cooperative groups* per realizzare l'intero ciclo a punto fisso all'interno di un singolo kernel.

Il repository contiene nove implementazioni CUDA distinte dello stesso algoritmo logico, organizzate in due famiglie (*rule-driven* e *atom-driven*), più una baseline seriale su CPU usata come riferimento di correttezza. Come caso di studio per il testing su larga scala è stato usato il Sudoku, le cui celle e vincoli sono codificati come atomi e regole pesate.

Questa relazione descrive: la formalizzazione del problema (Sezione ??); il formato e la rappresentazione in memoria dei dati di input (Sezione ??); le diverse strategie di parallelizzazione adottate (Sezione ??); la metodologia di testing (Sezione ??); e infine i risultati di performance raccolti sul benchmark Sudoku (Sezione ??).

2 Descrizione del problema

2.1 Regole pesate

Sia $\mathcal{A}$ un insieme finito di atomi proposizionali. Si considera un frammento di linguaggio logico costituito da sole regole della forma

$$h \leftrightarrow B \leq \{L_1 : w_1, \dots, L_k : w_k\}, \quad (1)$$

dove h è la *head* della regola e il *body* viene denotato con $W \equiv B \leq \{L_1 : w_1, \dots, L_k : w_k\}$. Sia h che $L_1, \dots, L_k$ sono letterali proposizionali ground (atomi o loro negazioni), mentre B (il *bound*) e i pesi $w_1, \dots, w_k$ sono interi positivi.

Dato un insieme di letterali M non contraddittorio, M soddisfa un letterale L ($M \models L$) se $L \in M$, lo falsifica se $L^c \in M$, altrimenti L è indeterminato. M è completo se per ogni atomo contiene esattamente uno tra l'atomo e la sua negazione.

Per ogni regola e per un dato M si definiscono le quantità

$$S_{sat} = \sum_{\substack{i \in \{1, \dots, k\} \\ M \models L_i}} w_i, \quad S_{undef} = \sum_{\substack{i \in \{1, \dots, k\} \\ L_i \text{ indeterminato}}} w_i, \quad S_{max} = S_{sat} + S_{undef}.$$

Il body W è vero se $B \leq S_{sat}$ (cioè se il peso dei letterali già soddisfatti raggiunge il bound), e questa nozione è ben definita anche quando alcuni L_i sono ancora indeterminati. M soddisfa la regola $h \leftrightarrow W$ se soddisfa la bi-implicazione.

2.2 Propagazione

Dato un insieme P di regole e un'assegnazione (eventualmente parziale) M , l'obiettivo è estendere M sfruttando le regole di P , in modo che nessuna regola venga falsificata. Le due direzioni di propagazione sono:

Body $\rightarrow$ Head. Se $S_{sat} \geq B$ allora h deve essere posto vero; se $S_{max} < B$ allora h deve essere posto falso. Se h risulta già assegnato al valore complementare si ha una contraddizione.

Head $\rightarrow$ Body. Se $M \models h$ allora W deve essere soddisfatto: per ogni L_i indeterminato, L_i va propagato (posto vero) esattamente quando $S_{max} - w_i < B$, perché altrimenti nemmeno rendendo veri tutti gli altri letterali indeterminati si raggiungerebbe il bound. Simmetricamente, se $M \models h^c$ allora W non può essere soddisfatto: per ogni L_i indeterminato, L_i^c va propagato esattamente quando $S_{sat} + w_i \geq B$, perché altrimenti rendere vero L_i farebbe superare il bound, violando la regola.

L'inserimento di un nuovo letterale in M può abilitare ulteriori propagazioni: il processo deve quindi essere iterato fino a un punto fisso, cioè finché non ci sono più variazioni in M oppure finché non si rileva una contraddizione.

2.3 Obiettivo del progetto

Il propagatore CUDA deve calcolare, a partire da P e da un M iniziale parziale, l'estensione massimale $\hat{M}$ di M che non falsifica alcuna regola di P (oppure segnalare un fallimento se tale $\hat{M}$ non esiste), e classificare ogni regola come soddisfatta, violata o indeterminata rispetto a $\hat{M}$. Tutto ciò deve avvenire con un grado di parallelismo che sfrutti la GPU su istanze di dimensione limitata solo dalla memoria del device.

3 Rappresentazione e memorizzazione dei dati

3.1 Formato di input testuale

L'input è un testo in stile DIMACS, letto da standard input dal parser (`src/parser.cpp`, dichiarato in `include/parser.hpp`):

- una riga `p <num_atomi> <num_regole>` di intestazione;
- `num_regole` righe `r <head> <bound> <k> <lit_1> <w_1> ... <lit_k> <w_k>`, una per regola, dove `head` è un letterale (positivo = atomo, negativo = sua negazione) e i `k` letterali del body sono dati come coppie letterale/peso;
- una riga `a <lit_1> <lit_2> ... 0` con l'assegnazione iniziale (i fatti noti), terminata da 0.

Ad esempio, il test `tests/input/test1.in`:

```

1 p 4 2
2 r 3 5 2 1 3 -2 4
3 r -4 2 2 3 2 2 1
4 a 1 -2 0

```

descrive 4 atomi e 2 regole: la regola 0 ha head sull'atomo 3, bound 5 e body $\{1 : 3, -2 : 4\}$; la regola 1 ha head $\neg 4$, bound 2 e body $\{3 : 2, 2 : 1\}$; l'assegnazione iniziale pone l'atomo 1 vero e l'atomo 2 falso. Il parser legge i token con `cin`, popolando direttamente le strutture descritte di seguito; non viene costruita alcuna rappresentazione ad albero o a lista delle regole.

3.2 Struttura DIMACSInput: layout a vettori piatti (CSR)

Il parser produce la struttura DIMACSInput (`include/parser.hpp`):

```

1 enum TruthValue { FALSE = 0, UNDEF = -1, TRUE = 1 };
2
3 struct DIMACSInput {
4     int num_atoms;
5     int num_rules;
6
7     vector<int> M;           // M[0] inutilizzato, M[1..num_atoms]
8     vector<int> head;       // head[r] = letterale head della regola r
9     vector<int> bound;      // bound[r] = bound B della regola r
10    vector<int> rule_offsets; // offset di inizio body, size = num_rules+1
11    vector<int> flat_lits;   // letterali di tutti i body, concatenati
12    vector<int> flat_weights; // pesi corrispondenti a flat_lits
13 };

```

Le regole non sono memorizzate come oggetti separati con un proprio vettore di letterali: i body di tutte le regole vengono concatenati in due vettori piatti, `flat_lits` e `flat_weights`, mentre `rule_offsets` indica, per ogni regola, l'intervallo [`rule_offsets[r]`, `rule_offsets[r+1]`) che ne delimita il body all'interno dei vettori piatti. Si tratta dello stesso principio della rappresentazione *Compressed Sparse Row* usata per matrici sparse: evita puntatori e liste concatenate, garantisce che l'intera base di regole sia contenuta in un numero fisso di array contigui di interi, e rende immediato sia il trasferimento Host→Device (un'unica `cudaMemcpy` per array) sia l'accesso coalescente da parte dei thread CUDA, che possono iterare sul body di una regola con un semplice ciclo su un intervallo di indici.

Il vettore `M` rappresenta lo stato di verità degli atomi (`FALSE = 0`, `UNDEF = -1`, `TRUE = 1`) ed è l'unica struttura mutabile durante la propagazione: sia le strategie rule-driven che quelle atom-driven operano leggendo e scrivendo concorrentemente su `M` (o su una sua copia locale), tipicamente tramite operazioni atomiche.

3.3 Tabelle inverse per le varianti atom-driven

Le strategie atom-driven, descritte in Sezione ??, devono poter risalire rapidamente, dato un atomo modificato, all'insieme delle regole in cui esso compare. A questo scopo viene costruita, lato host, la struttura `ReverseTables` (`include/reverse_tables.hpp`, popolata da `src/reverse_tables.cpp`):

```

1 struct ReverseTables {
2     vector<int> atom_body_offsets; // CSR: regole/letterali/pesi in cui
3     vector<int> atom_body_rules;   // l'atomo compare nel BODY
4     vector<int> atom_body_lits;
5     vector<int> atom_body_weights;
6
7     vector<int> atom_head_offsets; // CSR: regole in cui l'atomo
8     vector<int> atom_head_rules;   // compare come HEAD
9 };

```

Anche qui il layout è CSR, analogo a quello di `DIMACSInput`: per ogni atomo si conosce, tramite gli offset, l'intervallo di regole/letterali/pesi associati. Questa struttura è l'analogo, per gli atomi, di un indice invertito: la rappresentazione diretta (regola → letterali) non permette di rispondere efficientemente alla domanda “in quali regole appare questo atomo”, che è invece l'operazione centrale delle strategie atom-driven.

3.4 Allocazione sul device

Tutte le strutture vengono trasferite sulla GPU come array paralleli indipendenti tramite `cudaMalloc/cudaMemcpy` (es. `d_M`, `d_head`, `d_bound`, `d_rule_offsets`, `d_flat_lits`, `d_flat_weights`, e per le varianti atom-driven anche le tabelle inverse). A questi si aggiungono due flag globali

a un solo intero, `d_changed` e `d_contradiction`, usati per rilevare la convergenza al punto fisso e le contraddizioni senza dover ispezionare l'intero stato di `M` da host.

3.5 Formato di output

L'output (`src/printer.cpp`) è anch'esso testuale, su quattro righe:

```
1 s SUCCESS | CONTRADICTION | ERROR
2 v <letterali assegnati> 0
3 d <indici regole determinate, con segno> 0
4 u <indici regole indeterminate> 0
```

La riga `v` elenca i letterali di $\hat{M}$. La riga `d` elenca, per ogni regola completamente determinata, il suo indice con segno positivo se la regola risulta soddisfatta (head e body concordano) o negativo se risulta violata; la riga `u` elenca le regole per cui head o body sono ancora indeterminati in $\hat{M}$. Ad esempio, per `test1.in` l'output atteso è `s SUCCESS / v 1 -2 3 -4 0 / d 1 2 0 / u 0`: entrambe le regole risultano determinate e soddisfatte.

4 Strategie di soluzione

Tutte le nove varianti CUDA in `src/` implementano lo stesso algoritmo logico descritto in Sezione ??, ma differiscono nel modo in cui distribuiscono il lavoro tra i thread e nel modo in cui realizzano il ciclo di iterazione fino al punto fisso. Si possono classificare lungo due assi ortogonali: la *granularità del lavoro* (una regola per blocco, per tile, o guidata dagli atomi modificati) e il *meccanismo di iterazione* (ri-lancio del kernel da host, oppure ciclo a punto fisso interamente sul device tramite cooperative groups).

4.1 Rule-driven vs. atom-driven

Nelle varianti **rule-driven** (`rule*.cu`) ogni iterazione ricalcola S_{sat} , S_{undef} per *tutte* le regole, anche per quelle il cui body non è cambiato dall'iterazione precedente: è uno schema semplice, ma con lavoro ridondante quando solo pochi atomi cambiano valore.

Nelle varianti **atom-driven** (`atom*.cu`) si mantiene invece, per ogni regola, una somma parziale S_{sat}/S_{undef} aggiornata incrementalmente: quando un atomo viene assegnato, si usano le tabelle inverse per individuare solo le regole in cui l'atomo compare (nel body o come head) e si aggiornano atomicamente le loro somme, accodando le regole toccate per la successiva fase di deduzione. Il prezzo da pagare è la gestione di strutture aggiuntive (somme persistenti, coda di atomi modificati, tabelle inverse) e un pattern di accesso meno regolare.

4.2 Baseline: un blocco per regola/atomo, ciclo da host

`rule.cu` e `atom.cu` rappresentano le implementazioni di riferimento. In `rule.cu` ogni blocco CUDA elabora una regola (`blockIdx.x = rule_id`): i thread del blocco si dividono il body, calcolano somme parziali di S_{sat}/S_{undef} e le riducono in shared memory con una riduzione ad albero, poi il thread 0 applica la propagazione body→head e, dopo una `__syncthreads()`, tutti i thread applicano la propagazione head→body:

```
1 // Body -> Head (estratto da rule.cu)
2 if (threadIdx.x == 0) {
3     if (S_sat >= B) {
4         atomic_assign(M, h_atom, h_val, contradiction, changed);
5     } else if (S_max < B) {
6         atomic_assign(M, h_atom, h_not_val, contradiction, changed);
7     }
8 }
```

dove `atomic_assign` usa `atomicCAS` per assegnare un atomo solo se ancora indeterminato, segnalando una contraddizione se il vecchio valore differisce da quello che si vuole scrivere. Poiché un singolo lancio di kernel esegue un solo passo di propagazione, il ciclo a punto fisso è realizzato *da host*, ri-lanciando il kernel finché il flag `changed` resta a 1:

```

1 while (h_changed == 1 && h_contradiction == 0) {
2     cudaMemset(d_changed, 0, sizeof(int));
3     kernel<<<blocks_per_grid, THREADS_PER_BLOCK, shared_mem_size>>>(...);
4     cudaDeviceSynchronize();
5     cudaMemcpy(&h_changed, d_changed, sizeof(int), cudaMemcpyDeviceToHost);
6     cudaMemcpy(&h_contradiction, d_contradiction, sizeof(int),
7         cudaMemcpyDeviceToHost);
8 }

```

`atom.cu` segue lo stesso schema concettuale ma divide il lavoro in due kernel distinti (`init_sums_kernel`/`update_sums_kernel` per il mantenimento incrementale delle somme e `deduce_kernel` per la deduzione), con una coda di atomi modificati scambiata tra iterazioni (double buffering `d_queue_in`/`d_queue_out`) e un kernel di deduzione che salta le regole non toccate nell'iterazione corrente (`updated_rules[rule_id] == 0`).

4.3 Cooperative groups a livello di grid

`rule_cg.cu` e `atom_cg.cu` eliminano il ri-lancio ripetuto del kernel: usando `cg::grid_group` e la primitiva `grid.sync()`, l'intero ciclo a punto fisso viene eseguito *all'interno di un solo* `cudaLaunchCooperativeKernel`. In `rule_cg.cu` ogni blocco elabora più regole con un ciclo a stride sulla grid (`for (rule_id = blockIdx.x; ...; rule_id += gridDim.x)`), e tre `grid.sync()` per iterazione separano il reset del flag globale, l'elaborazione delle regole e il controllo di terminazione:

```

1 while (flag) {
2     if (threadIdx.x == 0 && blockIdx.x == 0) *changed = 0;
3     grid.sync();
4     for (int rule_id = blockIdx.x; rule_id < num_rules; rule_id += gridDim.x)
5     {
6         /* calcolo S_sat/S_undef, body->head, head->body, come in rule.cu */
7     }
8     grid.sync();
9     if (*changed == 0 || *contradiction == 1) flag = false;
10    grid.sync();
11 }

```

Il numero di blocchi lanciati viene scelto in base all'occupazione massima del device (`cudaOccupancyMaxActiveBlocksPerMultiprocessor` $\times$ numero di SM), requisito necessario perché un kernel cooperativo deve poter avere tutti i suoi blocchi residenti simultaneamente. Questo elimina l'overhead di ri-lancio del kernel e delle relative `cudaMemcpy` di sincronizzazione, al costo della sincronizzazione globale (più costosa di una semplice barriera di blocco) a ogni iterazione.

4.4 Packing a tile

`rule_packed.cu` e `atom_packed.cu` cambiano la granularità di parallelismo: invece di assegnare un intero blocco a una regola, il blocco viene partizionato in *tile* di `TILE_SIZE` thread (`cg::tiled_partition<TILE_SIZE>`, con `TILE_SIZE = 16` nel codice), e ogni tile elabora una regola:

```

1 cg::thread_block_tile<TILE_SIZE> tile = cg::tiled_partition<TILE_SIZE>(block);
2 int rule_id = (blockIdx.x * tile.meta_group_size()) + tile.meta_group_rank();
3 ...

```

```

4 for (int offset = tile.size() / 2; offset > 0; offset /= 2) {
5     partial_S_sat += tile.shfl_down(partial_S_sat, offset);
6     partial_S_undef += tile.shfl_down(partial_S_undef, offset);
7 }

```

La riduzione delle somme parziali avviene quindi tramite `shfl_down` all'interno del warp/tile, senza passare per la shared memory né per barriere `__syncthreads()` a livello di blocco: per regole con body corto (come nel caso del Sudoku, dove ogni cella ha pochi vincoli) questo riduce sensibilmente l'overhead di sincronizzazione rispetto alla riduzione ad albero in shared memory usata da `rule.cu`, e aumenta il numero di regole elaborate in parallelo per blocco (più tile per blocco, anziché un blocco intero per regola). `rule_packed_cg.cu` e `atom_packed_cg.cu` combinano questo schema di packing con il ciclo a punto fisso via `grid.sync()` descritto nella sezione precedente, ottenendo entrambi i vantaggi.

4.5 Variante avanzata: cache locale e punto fisso a due livelli

`rule_partial_fp_cg.cu` è la variante più sofisticata. Oltre al packing a tile e alla sincronizzazione di grid, introduce una copia locale di `M` in shared memory (`M_local`) per il sottoinsieme di regole assegnato a ciascun blocco, e un *doppio ciclo* a punto fisso:

- un ciclo **interno** (`flag_local`) che itera la propagazione `body→head` e `head→body` usando esclusivamente `M_local` e i letterali del blocco, già copiati in shared memory (`lits_local`, `weights_local`), fino a che il blocco non raggiunge un punto fisso locale o una contraddizione locale;
- un ciclo **esterno** (`flag_global`) che, dopo ogni punto fisso locale, ripropaga in `M` globale solo gli atomi effettivamente assegnati in `M_local`, e sincronizza tutti i blocchi con `grid.sync()` prima di rileggere lo stato globale aggiornato e ripetere.

```

1 bool flag_global = true;
2 while (flag_global) {
3     if (blockIdx.x == 0 && threadIdx.x == 0) *changed = 0;
4     grid.sync();
5     for (int i = block.thread_rank(); i < num_atoms + 1; i += block.size())
6         M_local[i] = M[i]; // copia M globale in shared memory
7     block.sync();
8
9     bool flag_local = true;
10    while (flag_local) {
11        /* body->head e head->body su M_local, riduzione di tile via shfl_down */
12        block.sync();
13        if (*local_changed == 0 || *local_contradiction == 1) flag_local = false;
14    }
15
16    for (int i = block.thread_rank(); i < num_atoms + 1; i += block.size())
17        if (M_local[i] != UNDEF)
18            atomic_assign(M, i, M_local[i], contradiction, changed); //
19            // riporta in M globale
20    grid.sync();
21    if (*changed == 0 || *contradiction == 1) flag_global = false;
22 }

```

L'idea è di concentrare quante più iterazioni possibile dentro un blocco, usando solo shared memory e barriere di blocco (rapide), e di ricorrere alla sincronizzazione globale (più lenta) solo quando serve davvero propagare informazione tra blocchi diversi: questo dovrebbe

ridurre il traffico verso la memoria globale rispetto alle varianti `*_cg` pure, che invece leggono/scrivono direttamente su `M` globale a ogni passo. La shared memory necessaria viene calcolata e allocata dinamicamente all'avvio (`max_shared_mem * sizeof(int)` passato a `cudaLaunchCooperativeKernel`), in funzione del numero di atomi e del numero di letterali assegnati al blocco.

4.6 Riepilogo

Variante	Driver	Sincronizzazione	Granularità
<code>rule / atom</code>	<code>rule / atom</code>	ri-lancio da host	blocco per regola/atomo
<code>rule_cg / atom_cg</code>	<code>rule / atom</code>	<code>grid.sync()</code>	blocco per regola/atomo
<code>rule_packed / atom_packed</code>	<code>rule / atom</code>	ri-lancio da host	tile per regola
<code>rule_packed_cg / atom_packed_cg</code>	<code>rule / atom</code>	<code>grid.sync()</code>	tile per regola
<code>rule_partial_fp_cg</code>	<code>rule</code>	<code>grid.sync()</code> + cache locale	tile per regola, punto fisso a 2 liv

Tabella 1: Le nove varianti CUDA classificate per famiglia, meccanismo di sincronizzazione e granularità.

5 Testing

5.1 Test unitari

In `tests/input/` sono presenti 11 istanze DIMACS minime, ciascuna mirata a un caso preciso della semantica di propagazione descritta in Sezione ??, con l'output atteso corrispondente in `tests/results/`:

- `test1/2/3.in` – casi base di propagazione `body→head` e `head→body`;
- `test_bodyhead.in / test_bodyhead_fail.in` – propagazione `body→head`, nel caso normale e nel caso che porta a contraddizione;
- `test_headbody.in / test_headbody_fail.in` – propagazione `head→body`, casi normale e di fallimento;
- `test_fixedpoint.in` – propagazione a catena su più regole, che richiede più iterazioni prima di raggiungere il punto fisso;
- `test_contradiction.in` – assegnazione iniziale che, propagata, porta a una contraddizione diretta;
- `test_neglet.in` – corretta gestione di letterali negati nel `body`;
- `test_noprop.in` – caso in cui nessuna propagazione è possibile (l'assegnazione resta quella iniziale).

Ogni eseguibile (uno per variante, prodotto dal `Makefile`) viene lanciato con un file di test in ingresso su standard input, e l'output prodotto viene confrontato testualmente con il file di risultato atteso. Poiché tutte le varianti implementano lo stesso algoritmo logico, questi test verificano la correttezza semantica indipendentemente dalla particolare strategia di parallelizzazione: un eventuale bug di sincronizzazione (race condition, propagazione mancata o duplicata) emerge come scostamento dall'output di riferimento.

5.2 Baseline seriale

`benchmarks/serial_naive.cpp` implementa lo stesso algoritmo in C++ puro, a singolo thread, e viene usato come secondo riferimento di correttezza indipendente dalle strutture dati GPU-specifiche, oltre che come termine di paragone “limite” per valutare lo speedup ottenibile dalle versioni parallele.

5.3 Benchmark su larga scala: Sudoku

Per un test di carico più realistico, il progetto codifica il Sudoku come istanza del propagatore: `tests/sudoku.py` genera, per ogni puzzle, le regole che codificano i vincoli di riga, colonna e blocco 3×3 (ogni cella è un insieme di atomi, uno per ogni valore possibile, con regole che impongono l’unicità), e i valori già noti del puzzle diventano l’assegnazione iniziale `a`. I dataset `sudoku_small.csv`, `sudoku_medium.csv` e `sudoku.csv` contengono puzzle in formato compatto (stringa di 81 caratteri), convertiti nel formato DIMACS del propagatore da `compact_to_dimacs.py`, e l’output del propagatore viene riconvertito nel formato compatto da `dimacs_to_compact.py` per il confronto con la soluzione nota.

Lo script `run_tests.sh` automatizza l’intero ciclo per un dato eseguibile:

1. converte il dataset CSV in istanze DIMACS (`tests/sudoku/input/*.in`);
2. per ciascuna istanza, esegue l’eseguibile sotto `nsys profile -t nvtx,cuda`, salvando il profilo (`.nsys-rep`) e le statistiche di kernel/memoria/API CUDA in CSV (`nsys stats -report=cuda_gpu_kern_sum,cuda_gpu_mem_size_sum,cuda_api_sum`);
3. confronta l’output convertito con la soluzione attesa tramite `diff -w`, contando test passati e falliti;
4. accumula le metriche di profiling di tutte le istanze in un unico CSV per variante (`tests/sudoku/stats/<variante>_nsys_summary.csv`).

Questo combina, nello stesso test, una verifica di correttezza (il puzzle risolto deve coincidere con la soluzione nota) e una raccolta sistematica di metriche di performance per ogni variante, che viene poi aggregata (script `analyze.py`) nella tabella e nei grafici discussi nella sezione seguente.

6 Risultati delle performance

Le misure sono state raccolte profilando con `nsys` le nove varianti sulle 9 istanze di test del Sudoku 9×9 (731 atomi, 648 regole per istanza), mediando i tempi su tutte le istanze. La Tabella ?? riporta, per ciascuna configurazione, il tempo di kernel CUDA, il tempo trascorso in trasferimenti di memoria, il tempo di `cudaMalloc`, il tempo lato CPU e il tempo totale.

6.1 Discussione

Il primo dato evidente è che, su istanze piccole come il Sudoku 9×9 , il tempo di `cudaMalloc` domina il tempo totale (oltre il 98% in ogni configurazione): l’allocazione di memoria sul device ha un costo fisso che non dipende dalla strategia di propagazione, e per istanze di queste dimensioni satura completamente il confronto se si guarda solo al tempo totale. Il confronto significativo tra le strategie va quindi fatto sul *tempo di kernel*, dove le differenze tra le varianti emergono chiaramente (da circa $30 \mu\text{s}$ a oltre $105 \mu\text{s}$).

Su questa metrica, `rule_packed` risulta il più rapido, seguito da `rule_packed_cg`: il packing a tile, che riduce regole piccole (come quelle del Sudoku, con pochi letterali nel body) a un

Configurazione	Kernel (ns)	Memoria (ns)	Malloc (ns)	CPU (ns)	Totale (ns)
rule_packed	29 909	267 070	1.620×10^8	107 043	1.624×10^8
rule_packed_cg	36 951	113 656	1.629×10^8	39 208	1.631×10^8
rule_cg	75 231	114 974	1.631×10^8	77 717	1.633×10^8
atom_packed_cg	66 329	169 887	1.632×10^8	146 158	1.636×10^8
atom_cg	105 407	167 846	1.632×10^8	178 041	1.637×10^8
atom_packed	55 707	317 061	1.632×10^8	245 741	1.638×10^8
rule_partial_fp_cg	55 085	116 655	1.636×10^8	108 769	1.639×10^8
rule	60 925	220 550	1.640×10^8	122 600	1.644×10^8
atom	70 957	317 256	1.652×10^8	261 722	1.658×10^8

Tabella 2: Tempi medi (ns) sulle 9 istanze del benchmark Sudoku 9×9 , ordinati per tempo totale crescente.

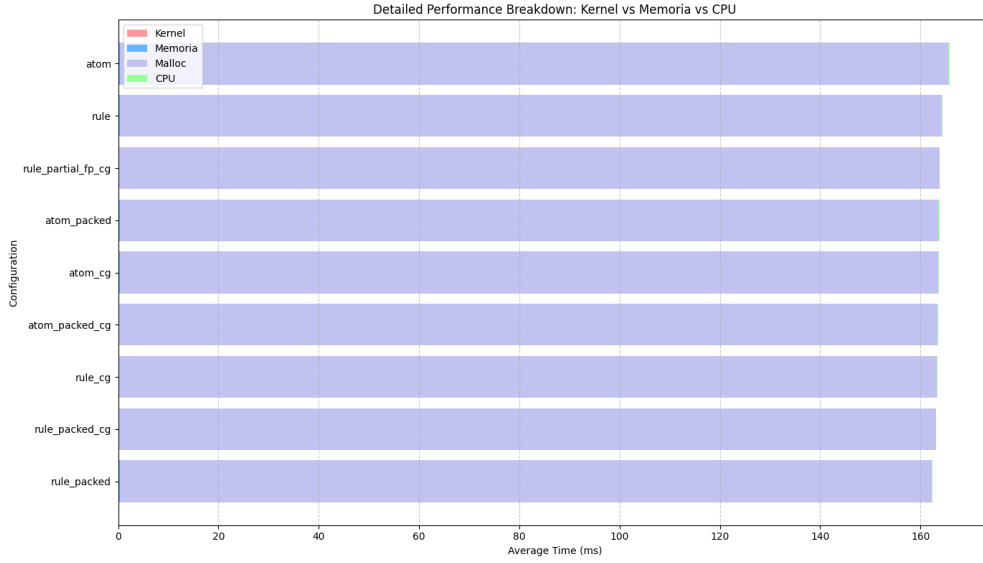


Figura 1: Confronto del tempo totale (incluso `cudaMalloc`) tra le nove varianti.

singolo warp/tile con riduzione via `shfl_down`, si rivela la leva di ottimizzazione più efficace, più della sincronizzazione globale via cooperative groups. Le due varianti packed restano le migliori anche guardando il tempo totale (Tabella ??), grazie anche a un tempo di `cudaMalloc` leggermente inferiore.

Le varianti `*_cg` senza packing (`rule_cg`, `atom_cg`) mostrano un kernel time superiore alla baseline non cooperativa corrispondente nel caso `atom_cg`, segno che il costo della sincronizzazione di grid (`grid.sync()`, eseguita più volte per iterazione) non viene sempre ripagato dal risparmio sui ri-lanci di kernel, soprattutto quando il numero di iterazioni a punto fisso è già piccolo, come nel caso del Sudoku.

La variante avanzata `rule_partial_fp_cg` ottiene un kernel time intermedio ($55 \mu\text{s}$, sostanzialmente alla pari con `atom_packed`), nonostante introduca caching locale e un ciclo a punto fisso a due livelli pensato per ridurre il traffico su memoria globale: su istanze piccole come queste, l'overhead di gestione della shared memory dinamica e della copia iniziale di M nel blocco non viene compensato da un numero di iterazioni sufficientemente alto da rendere vantaggioso il caching, il cui beneficio è presumibilmente più marcato su istanze con basi di regole molto più grandi o con molte più iterazioni di punto fisso.

In generale, le varianti **atom-driven** risultano sistematicamente più lente delle corrispondenti varianti **rule-driven** (confrontando `atom` vs. `rule`, `atom_cg` vs. `rule_cg`, `atom_packed`

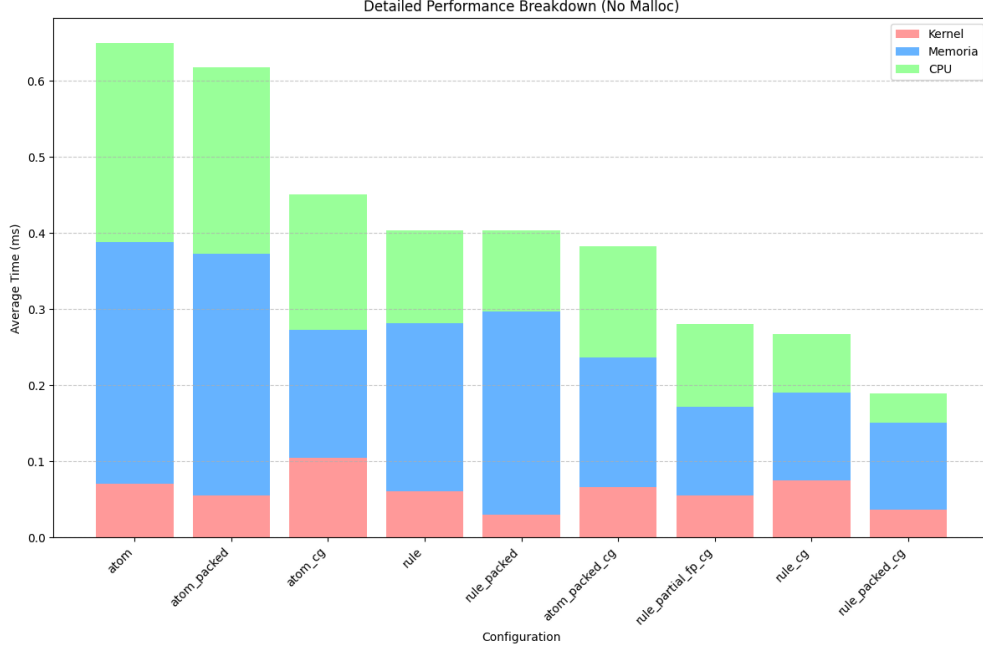


Figura 2: Confronto escludendo il tempo di `cudaMalloc`, per isolare il costo effettivo di kernel e trasferimenti.

vs. `rule_packed`): il costo dell'indirizzamento tramite le tabelle inverse e della gestione della coda di atomi modificati supera, su istanze di queste dimensioni, il risparmio di lavoro ridondante che lo schema atom-driven dovrebbe in teoria offrire rispetto al ricalcolo completo delle somme a ogni iterazione fatto dalle varianti rule-driven.

7 Conclusioni

Tutte le nove varianti CUDA producono lo stesso risultato logico, verificato sia sui test unitari mirati sia sul benchmark Sudoku confrontando con le soluzioni note: le diverse strategie di parallelizzazione sono quindi intercambiabili dal punto di vista della correttezza, e si distinguono solo per le prestazioni.

Sul benchmark utilizzato, il packing a tile (`rule_packed`/`rule_packed_cg`) è risultata la scelta più efficace, più dell'uso di cooperative groups a livello di grid o del caching locale in shared memory introdotto dalla variante più sofisticata (`rule_partial_fp_cg`). Questo è in parte attribuibile alle dimensioni contenute delle istanze di test (regole con pochi letterali nel body, poche iterazioni di punto fisso necessarie), che non permettono di ammortizzare l'overhead delle tecniche più sofisticate. Anche il costo fisso di `cudaMalloc`, dominante sul tempo totale per istanze di queste dimensioni, indica che un confronto più informativo richiederebbe istanze sensibilmente più grandi, oppure il batching di più puzzle in un unico lancio di kernel per ammortizzare i costi fissi di allocazione e per dare alle strategie con overhead di sincronizzazione più alto (cooperative groups, caching) margine per ripagarsi su un numero maggiore di iterazioni utili.